JavaScript Assignments

Naveen AutomationLabs | 2026 Playwright Batch G1

Session 01 - What is JavaScript, Node.js, VSC, V8 Engine, Browser Engines

Covers: Introduction to JavaScript, setting up Node.js and VS Code, V8 engine basics, browser engines.

Q1. What is JavaScript? Is it a compiled language or an interpreted language?

Q2. Name at least three places where JavaScript can run.

Q3. What is Node.js? Why do we need it if we already have browsers?

Q4. What is the V8 engine? Which browser uses it?

Q5. Name the JavaScript engines used by Chrome, Firefox, and Safari.

Q6. What is the difference between a browser engine and a JavaScript engine?

Q7. How do you check if Node.js is installed on your machine? Write the command.

Q8. What is VS Code? Name two extensions that are useful for writing JavaScript.

Q9. Write a simple program that prints 'Hello, JavaScript!' using console.log and run it using Node.js.

Q10. Can JavaScript run without a browser? Explain how.

Session 02 - var, let, const, Hoisting, Primitive Data Types, Size, Ranges

Covers: Variable declarations, hoisting behavior, primitive types (string, number, boolean, null, undefined, symbol, bigint).

Q11. What are the three ways to declare a variable in JavaScript?

Q12. What is the difference between let and const?

Q13. Can you reassign a value to a const variable? What happens if you try?

Q14. What is hoisting? Give a simple example using var.

Q15. What happens if you try to use a let variable before declaring it?

Q16. List all the primitive data types in JavaScript.

Q17. What is the difference between null and undefined?

Q18. What does typeof return for: 42, 'hello', true, null, undefined?

Q19. What is the range of the Number type in JavaScript? What happens if you go beyond that range?

Q20. Write a small code to declare one variable with var, one with let, and one with const. Print all three.

Session 03 - Template Literals, Backtick, Math Operators, Concatenation

Covers: String concatenation, template literals with backticks, arithmetic and assignment operators.

Q21. What is string concatenation? Give an example using the + operator.

Q22. What are template literals? How do you create one?

Q23. Rewrite this using a template literal: 'My name is ' + name + ' and I am ' + age + ' years old.'

Q24. What is the difference between single quotes, double quotes, and backticks in JavaScript?

Q25. List all the basic math operators in JavaScript.

Q26. What is the result of: 10 % 3? Explain what the % operator does.

Q27. What is the result of: 2 ** 4?

Q28. What does the += operator do? Give an example.

Q29. What happens when you do: 'Hello' + 5? What type is the result?

Q30. Write a program that takes two numbers, adds them, and prints the result using a template literal.

Session 04 - Loose/Strict Equality, Infinity, NaN, Incremental/Decremental Operators

Covers: == vs ===, NaN, Infinity, ++, -- operators, type coercion.

Q31. What is the difference between == and ===?

Q32. What is the result of: 5 == '5'? What about 5 === '5'? Explain why.

Q33. What is NaN in JavaScript? Give one example that produces NaN.

Q34. How do you check if a value is NaN? Can you use === for this?

Q35. What is Infinity in JavaScript? What operation produces it?

Q36. What is the result of: Infinity + 100?

Q37. Explain the difference between i++ and ++i with an example.

Q38. What is the result of: let x = 5; let y = x++; What are the values of x and y?

Q39. What is type coercion? Give one example.

Q40. What is the result of: null == undefined? What about null === undefined?

Session 05 - Conditional Operators, if/else, else if, Switch Case, Debugging in VSC

Covers: if, else, else if, switch/case, ternary operator, debugging basics in VS Code.

Q41. Write the basic syntax of an if-else statement in JavaScript.

Q42. What is the purpose of 'else if'? When do you use it?

Q43. Write a program that checks if a number is positive, negative, or zero using if/else if/else.

Q44. What is a ternary operator? Write an example.

Q45. Write the syntax of a switch-case statement. What happens if you forget the break keyword?

Q46. Write a switch-case program that prints the day name based on a number (1 = Monday, 2 = Tuesday, etc.).

Q47. What is the default case in a switch statement? Is it required?

Q48. How do you set a breakpoint in VS Code for debugging JavaScript?

Q49. What is the difference between if-else and switch-case? When would you prefer one over the other?

Q50. Write a program using the ternary operator to check if a person's age is 18 or above and print 'Eligible' or 'Not Eligible'.

Session 06 - Loops: while, for, do-while, Truthy/Falsy Values

Covers: for, while, do-while loops, break, continue, truthy and falsy values.

Q51. Write the syntax of a for loop. What are the three parts inside the parentheses?

Q52. Write a for loop that prints numbers from 1 to 10.

Q53. What is the difference between a while loop and a do-while loop?

Q54. Write a while loop that prints numbers from 10 down to 1.

Q55. What do the break and continue keywords do inside a loop?

Q56. Write a for loop that prints only even numbers from 1 to 20.

Q57. What are falsy values in JavaScript? List all of them.

Q58. What are truthy values? Give three examples.

Q59. Write a program using a for loop to calculate the sum of numbers from 1 to 100.

Q60. What happens if you write a while loop without a condition that becomes false? How do you avoid it?

Session 07 - Arrays, Array Methods: pop, push, shift, unshift, splice, indexOf

Covers: Array creation, accessing elements, common array methods.

Q61. How do you create an array in JavaScript? Give an example with 5 elements.

Q62. How do you access the third element of an array? What index does it have?

Q63. What does the push() method do? Give an example.

Q64. What does the pop() method do? What does it return?

Q65. What is the difference between shift() and unshift()?

Q66. What does the splice() method do? Write an example that removes 2 elements starting from index 1.

Q67. How do you find the index of an element in an array? What does it return if the element is not found?

Q68. What is the length property of an array? How do you use it?

Q69. Write a program that creates an array of 5 fruits, adds one at the end, removes one from the beginning, and prints the final array.

Q70. What is the difference between splice() and slice()?

Answer Key

Session 01 - What is JavaScript, Node.js, VSC, V8 Engine, Browser Engines

Q1. JavaScript is a high-level, interpreted (or just-in-time compiled) programming language. It is primarily interpreted, though modern engines use JIT compilation for performance.

Q2. JavaScript can run in web browsers, on servers using Node.js, and in mobile apps (e.g., React Native).

Q3. Node.js is a runtime environment that lets you run JavaScript outside the browser, on servers and desktops. We need it to build backend applications and run JS scripts locally.

Q4. V8 is an open-source JavaScript engine developed by Google. It is used by Google Chrome and Node.js.

Q5. Chrome uses V8, Firefox uses SpiderMonkey, and Safari uses JavaScriptCore (also called Nitro).

Q6. A browser engine (like Blink or Gecko) handles rendering HTML and CSS. A JavaScript engine (like V8 or SpiderMonkey) specifically executes JavaScript code.

Q7. Run 'node --version' or 'node -v' in the terminal. If installed, it prints the version number.

Q8. VS Code (Visual Studio Code) is a free, lightweight code editor. Useful extensions: ESLint and Prettier.

Q9. console.log('Hello, JavaScript!'); -- Save as a .js file and run with: node filename.js

Q10. Yes. Using Node.js, JavaScript can run on servers, desktops, and command-line tools without needing a browser.

Session 02 - var, let, const, Hoisting, Primitive Data Types, Size, Ranges

Q11. var, let, and const.

Q12. let allows reassignment of value. const does not allow reassignment after the initial value is set.

Q13. No. Trying to reassign a const variable throws a TypeError: Assignment to constant variable.

Q14. Hoisting means variable and function declarations are moved to the top of their scope during compilation. Example: console.log(x); var x = 5; // prints undefined, not an error.

Q15. You get a ReferenceError. let variables are hoisted but not initialized, so they are in a 'temporal dead zone' until the declaration line.

Q16. string, number, boolean, null, undefined, symbol, and bigint.

Q17. undefined means a variable has been declared but no value has been assigned. null is an intentional assignment meaning 'no value' or 'empty'.

Q18. typeof 42 = 'number', typeof 'hello' = 'string', typeof true = 'boolean', typeof null = 'object' (this is a known JS quirk), typeof undefined = 'undefined'.

Q19. The safe integer range is -(2^53 - 1) to (2^53 - 1). Beyond that, precision is lost. You can use BigInt for larger numbers.

Q20. var a = 10; let b = 20; const c = 30; console.log(a, b, c);

Session 03 - Template Literals, Backtick, Math Operators, Concatenation

Q21. String concatenation joins two or more strings. Example: 'Hello' + ' ' + 'World' gives 'Hello World'.

Q22. Template literals are strings created with backticks (`) that support embedded expressions using ${expression}.

Q23. `My name is ${name} and I am ${age} years old.`

Q24. Single quotes and double quotes work the same way for basic strings. Backticks create template literals that allow multi-line strings and expression embedding with ${}.

Q25. +, -, *, /, % (modulus), ** (exponentiation).

Q26. 10 % 3 = 1. The % (modulus) operator returns the remainder of a division.

Q27. 2 ** 4 = 16. The ** operator is the exponentiation operator (2 to the power of 4).

Q28. += adds the right value to the variable and assigns the result. Example: let x = 5; x += 3; // x is now 8.

Q29. 'Hello5' -- JavaScript converts 5 to a string and concatenates. The result type is string.

Q30. let a = 10; let b = 20; let sum = a + b; console.log(`The sum of ${a} and ${b} is ${sum}`);

Session 04 - Loose/Strict Equality, Infinity, NaN, Incremental/Decremental Operators

Q31. == checks value equality with type coercion. === checks both value and type without coercion (strict equality).

Q32. 5 == '5' is true (type coercion converts '5' to number). 5 === '5' is false (different types: number vs string).

Q33. NaN stands for 'Not a Number'. Example: parseInt('hello') returns NaN. Also 0/0 gives NaN.

Q34. Use Number.isNaN(value) or isNaN(value). You cannot use === because NaN === NaN is false.

Q35. Infinity is a special numeric value representing something greater than any number. Example: 1 / 0 produces Infinity.

Q36. Infinity + 100 = Infinity. Any finite number added to Infinity is still Infinity.

Q37. i++ (post-increment) returns the current value first, then increments. ++i (pre-increment) increments first, then returns the new value.

Q38. y = 5, x = 6. Since x++ is post-increment, y gets the old value (5), then x becomes 6.

Q39. Type coercion is JavaScript automatically converting one type to another. Example: '5' - 2 = 3 (string '5' is converted to number).

Q40. null == undefined is true (they are loosely equal). null === undefined is false (different types).

Session 05 - Conditional Operators, if/else, else if, Switch Case, Debugging in VSC

Q41. if (condition) { // code } else { // code }

Q42. 'else if' lets you check multiple conditions one after another. Use it when you have more than two possible outcomes.

Q43. let n = -3; if (n > 0) { console.log('Positive'); } else if (n < 0) { console.log('Negative'); } else { console.log('Zero'); }

Q44. A ternary operator is a shorthand for if-else: condition ? valueIfTrue : valueIfFalse. Example: let result = (10 > 5) ? 'Yes' : 'No';

Q45. switch(expression) { case value1: // code; break; case value2: // code; break; default: // code } Without break, execution falls through to the next case.

Q46. let day = 3; switch(day) { case 1: console.log('Monday'); break; case 2: console.log('Tuesday'); break; case 3: console.log('Wednesday'); break; ... default: console.log('Invalid'); }

Q47. The default case runs when no other case matches. It is not required but is a good practice.

Q48. Click on the line number gutter in VS Code to set a red dot (breakpoint). Then run the debugger using the Run and Debug panel or press F5.

Q49. if-else is flexible and handles complex conditions and ranges. switch-case is cleaner when comparing one variable against many specific values. Use switch when you have many exact-match conditions.

Q50. let age = 20; let result = (age >= 18) ? 'Eligible' : 'Not Eligible'; console.log(result);

Session 06 - Loops: while, for, do-while, Truthy/Falsy Values

Q51. for (initialization; condition; update) { // code }. The three parts are: initialization (run once), condition (checked each iteration), update (run after each iteration).

Q52. for (let i = 1; i <= 10; i++) { console.log(i); }

Q53. A while loop checks the condition before running the code block. A do-while loop runs the code block at least once, then checks the condition.

Q54. let i = 10; while (i >= 1) { console.log(i); i--; }

Q55. break exits the loop completely. continue skips the current iteration and moves to the next one.

Q56. for (let i = 1; i <= 20; i++) { if (i % 2 === 0) { console.log(i); } }

Q57. false, 0, -0, '' (empty string), null, undefined, NaN. These are all the falsy values.

Q58. Truthy values are anything not falsy. Examples: 'hello', 42, true, [] (empty array), {} (empty object).

Q59. let sum = 0; for (let i = 1; i <= 100; i++) { sum += i; } console.log(sum); // 5050

Q60. It creates an infinite loop that never stops and may freeze or crash the program. Always make sure the condition will eventually become false or use a break statement.

Session 07 - Arrays, Array Methods: pop, push, shift, unshift, splice, indexOf

Q61. let arr = [10, 20, 30, 40, 50];

Q62. arr[2]. The index is 2 because arrays are zero-indexed (counting starts from 0).

Q63. push() adds one or more elements to the end of an array. Example: arr.push(60); // adds 60 at the end.

Q64. pop() removes the last element from an array and returns it. Example: let last = arr.pop();

Q65. shift() removes the first element from an array. unshift() adds one or more elements to the beginning of an array.

Q66. splice() can add or remove elements at any position. Example: arr.splice(1, 2); // removes 2 elements starting at index 1.

Q67. Use indexOf(). It returns the index of the element, or -1 if the element is not found. Example: arr.indexOf(30); // returns 2.

Q68. The length property returns the number of elements in an array. Usage: arr.length. Example: console.log(arr.length); // 5

Q69. let fruits = ['Apple', 'Banana', 'Cherry', 'Mango', 'Grape']; fruits.push('Orange'); fruits.shift(); console.log(fruits); // ['Banana', 'Cherry', 'Mango', 'Grape', 'Orange']

Q70. splice() modifies the original array by adding or removing elements. slice() returns a new array with a portion of the original without changing it. slice(1, 3) returns elements at index 1 and 2.